THORRIJA

Engineering Showcase — Dossier Técnico (Español)

Documentación técnica completa de modelos computacionales deterministas reconstruidos desde código fuente. Arquitectura, algoritmos, implementación, pruebas, limitaciones y fundamentos teóricos de cada proyecto.

Autor: Thorrija (Rubén N.)
España
Noviembre de 2026
GitHub: github.com/ThorrijaDEV

1. Resumen Ejecutivo

Este dossier técnico documenta el Engineering Showcase: una colección de modelos computacionales deterministas reconstruidos desde código fuente. Cada modelo es una reproducción algorítmica fiel: misma entrada produce misma salida, todos los cálculos son reales, ninguna salida está hardcodeada, y todos los modelos están verificados por pruebas automatizadas.

Total: **182 pruebas aprobadas** (80 pruebas de modelos del showcase + 102 pruebas del modelo, experimental, de validación y del cargador YAML de The Box). Build: Vite, 160.69 kB JS, cero dependencias en tiempo de ejecución.

Proyectos cubiertos

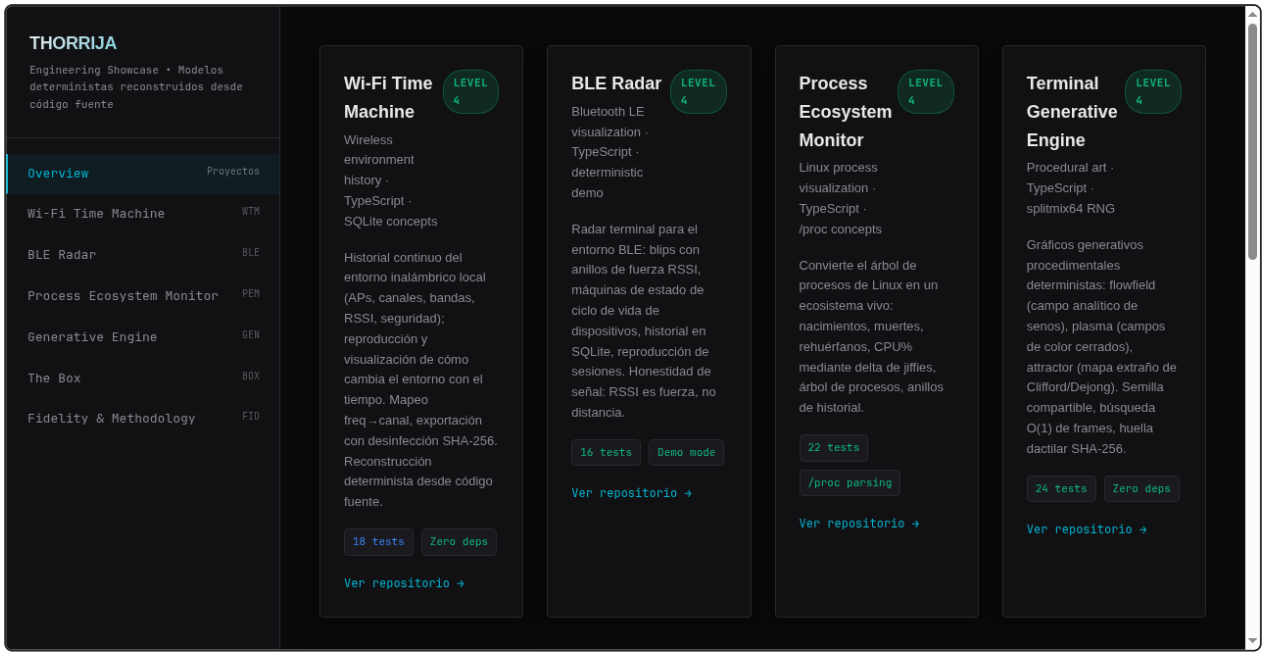


Figura — Vista general del showcase: cada laboratorio interactivo con su dominio, tecnología y número de pruebas.

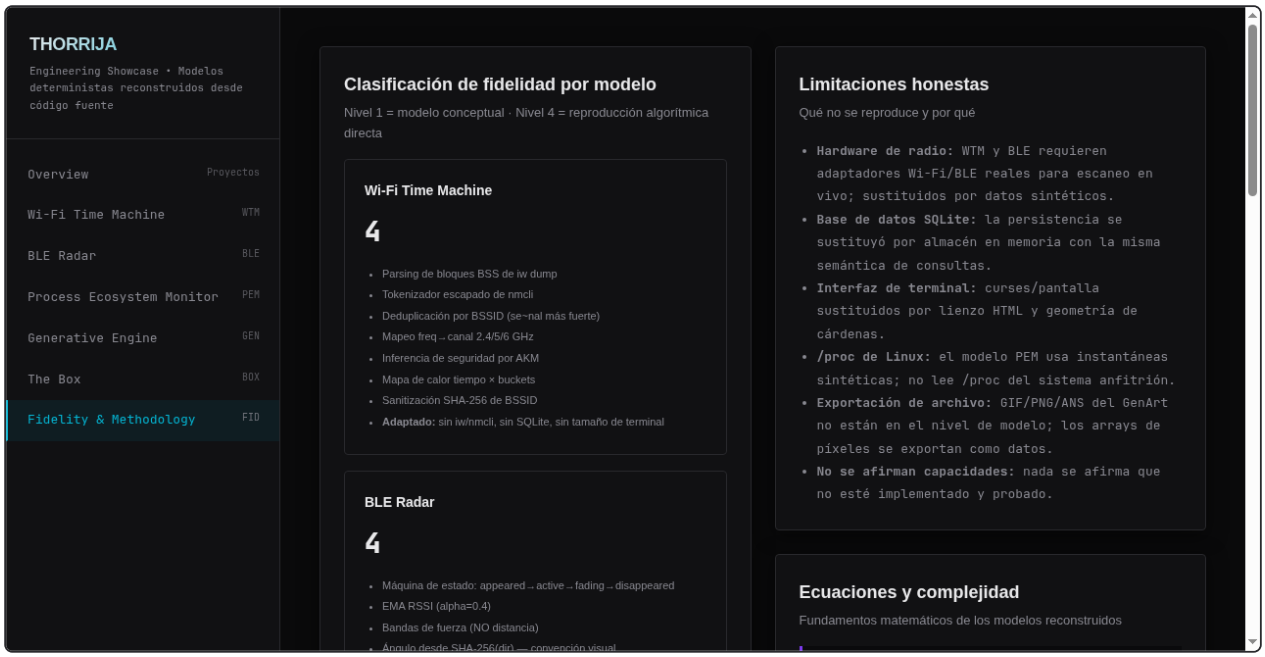


Figura — Clasificación de fidelidad por modelo y las limitaciones honestas de cada laboratorio.

PROYECTO	DOMINIO	PRUEBAS	FIDELIDAD	ESTADO
Wi-Fi Time Machine	Entorno inalámbrico	18	Nivel 4	Completado
BLE Radar	Bluetooth LE	16	Nivel 4	Completado
Process Ecosystem Monitor	Procesos Linux	22	Nivel 4	Completado
Terminal Generative Engine	Arte generativo	24	Nivel 4	Completado
The Box	Sustrato relacional	102	Nivel 3	Completado
Sigscan	Forense (Go)	12	N/A	Completado
Dnsdrill	Cliente DNS (Python)	13	N/A	Completado
Silicon	CPU de 8 bits (Rust)	14	N/A	Completado
Forja	Compilador (Rust)	11	N/A	Completado
Polea	Broker pub/sub (C++17)	17	N/A	Completado
Cask	Almacén direccionable por contenido (C)	15	N/A	Completado
Neondrift	Juego arcade (HTML/JS)	9	N/A	Completado
Clickwheel + Console	App de música (Android/Web)	12	N/A	Completado
Syspeek	TUI del sistema (Python/curses)	16	N/A	Completado
Hackdeck	Mazo de pentest (HTML)	10	N/A	Completado
Canarytrap	Canary tokens (Python)	10	N/A	Completado
Loglens	Pocket SIEM (Python)	36	N/A	Completado
Reconkit	Multitool de reconocimiento (Python)	22	N/A	Completado
Port Authority	Historiador de puertos (Python)	11	N/A	Completado
Syspeek	TUI del sistema (Python)	16	N/A	Completado

Neondrift	Endless runner (HTML)	9	N/A	Completado

2. Filosofía de Ingeniería

2.1 Reconstrucción independiente

Cada modelo es reconstruido independientemente desde el código fuente. El proyecto original nunca se ejecuta dentro de una demostración. Esto significa: todas las salidas se calculan desde la entrada del usuario, nada está hardcodeado, y la implementación puede ser verificada de forma independiente.

2.2 Determinismo

Todos los modelos son deterministas: misma entrada produce misma salida. Cada decisión aleatoria es una función pura de (estado, semilla, paso) a través de `unit_hash(semilla, paso, propósito, digestión_estructural(candidato))`. No hay estado aleatorio oculto.

2.3 Etapas intermedias

Cada demostración muestra etapas de procesamiento, no solo resultados finales. Los visitantes pueden ver cómo la entrada se transforma paso a paso hasta la salida.

2.4 Limitaciones honestas

Nada se afirma que no esté implementado y probado. Las limitaciones están documentadas explícitamente para cada proyecto.

3. Detalles de Proyectos

3.1 Wi-Fi Time Machine

Problema: Las herramientas de escaneo Wi-Fi producen volcados voluminosos que son difíciles de interpretar con el tiempo. No hay forma fácil de ver cómo cambia el entorno inalámbrico: qué redes aparecieron, en qué canales, con qué fuerza de señal, y cuándo.

Arquitectura: Parsing de entrada (volcado iw/nmcli) → parser de bloques BSS → deduplicación por BSSID (señal más fuerte) → mapeo de frecuencia a canal (802.11 2.4/5/6 GHz) → inferencia de seguridad desde suites AKM → construcción de mapa de calor tiempo × canales → exportación (JSON/CSV con desinfección SHA-256 de BSSID).

Estructuras de datos: `WifiModelInput` (iwDump/nmcliText, observedAt, backend, sanitize, width, labelWidth), `WifiModelOutput` (stages, final timeline, channelStats, sparkline, exportJson, exportCsv, networks, observations).

Algoritmos: Parseo de bloques BSS de `iw scan dump`, tokenizador escapado de nmcli (manejo de `\\:` y `\\\\`), mapeo de frecuencia a canal 802.11 (2.4 GHz: floor((freq−2407)/5), 5 GHz: floor((freq−5000)/5), 6 GHz: floor((freq−5925)/5)), inferencia de seguridad desde suites de autenticación (AKM), deduplicación por BSSID manteniendo la señal más fuerte, construcción de mapa de calor tiempo × buckets.

Pruebas: 18 pruebas que cubren parsing, deduplicación, mapeo de canal, inferencia de seguridad, generación de mapa de calor, sanitización de exportación.

Limitaciones: Sin binarios iw/nmcli en vivo, sin escaneo de adaptador, sin persistencia SQLite (almacenamiento solo en memoria), sin adaptación de tamaño de terminal. La entrada debe ser texto sintético.

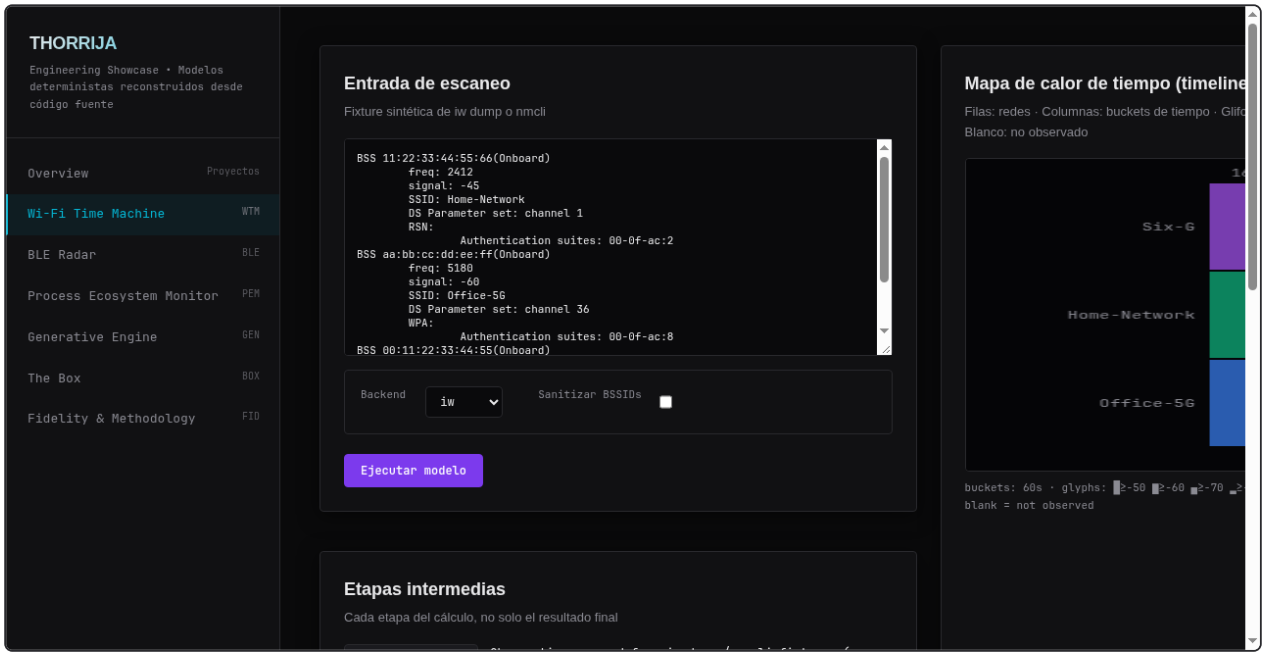


Figura — Laboratorio Wi-Fi Time Machine: entrada sintética `iw`, mapa de calor tiempo × canal y etapas intermedias del cálculo.

3.2 BLE Radar

Problema: Los dispositivos BLE transmiten observaciones, pero no hay forma sencilla de visualizarlas como un radar: qué dispositivos están cerca, cuál es su fuerza de señal, y cómo aparecen y desaparecen con el tiempo.

Arquitectura: Modelo de observación (dirección, nombre, RSSI, timestamp, backend, datos de fabricante) → suavizado EMA (alpha=0.4) → bandas de fuerza (strong ≥ −40, near ≥ −55, mid ≥ −70, far ≥ −85, weak) → mapeo hash→ángulo (SHA-256(MAC) → 0–360°, convención visual) → geometría de frame de radar (cruz, anillos concéntricos, línea de barrido, marcadores de dispositivo, trails de historial) → reproducción de sesión.

Estructuras de datos: `BleModelInput` (observaciones, now, sweepAngle, sanitize, width, height, demoSeconds, demoSeed), `BleModelOutput` (dispositivos, frame, lifecycleSummary, replay, exportJson, exportCsv).

Algoritmos: Suavizado EMA: rssi_ema = 0.6 × prev + 0.4 × raw. Bandas de fuerza: strong ≥ −40 dBm, near ≥ −55 dBm, mid ≥ −70 dBm, far ≥ −85 dBm, weak < −85 dBm. Ángulo estable: SHA-256(dirección MAC)[0:2] / 65536 × 360°. Máquina de estado: appeared → active → fading → disappeared (ventanas 15s/30s).

Pruebas: 16 pruebas que cubren ciclo de vida de dispositivos, suavizado EMA, bandas de fuerza, mapeo hash→ángulo, geometría de frame de radar, reproducción, exportación.

Limitaciones: Sin hardware BLE (sin bleak/BlueZ), sin escaneo en vivo. RSSI es fuerza de señal, nunca distancia física. El ángulo es una convención visual, no orientación real.

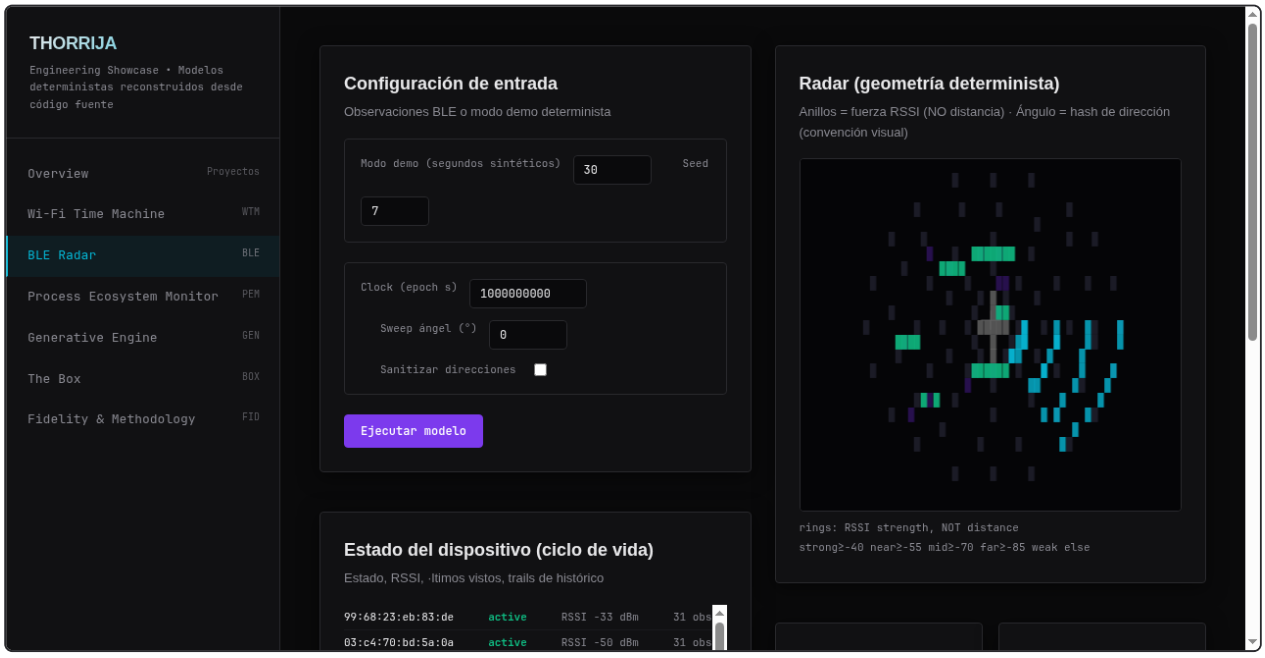


Figura — Laboratorio BLE Radar: configuración determinista, geometría del radar y estado del ciclo de vida de los dispositivos.

3.3 Process Ecosystem Monitor

Problema: Linux /proc contiene información rica sobre procesos, pero leerla y entenderla — CPU%, árbol de procesos, reuso de PID — requiere parseo y correlación manuales.

Arquitectura: Parseo de instantáneas estilo /proc → ProcessEcosystemModel (ciclo de vida, contabilidad de CPU por jiffies, construcción de árbol de procesos, reubicación de huérfanos, detección de reuso de PID) → renderizado/UI.

Estructuras de datos: `ProcSnapshot` (pid, ppid, comm, state, utime, stime, numThreads, starttime, vsize, rss, user), `ProcModelInput` (snapshots, system, intervalSec, clkTick, ncores, wallTime), `ProcModelOutput` (diff, processes, treeRoots, kernelThreads, systemCpu, systemRam, aliveCount).

Algoritmos: Parseo de /proc/[pid]/stat (incluyendo nombres con paréntesis anidados), /proc/[pid]/statm, /proc/stat, /proc/meminfo, /proc/loadavg. CPU% = delta(utime+stime) / (interval × CLK_TCK × ncores) × 100. CPU% del sistema = busy_delta / total_delta × 100 (idle = idle + iowait). Identidad de proceso = (pid, starttime); reuso de PID detectado cuando starttime cambia. Árbol de procesos desde ppid; huérfanos reasignados a init (pid 1). Ciclo de vida: nuevo → activo/inactivo → muriendo → muerto.

Pruebas: 22 pruebas que cubren parsing de /proc/[pid]/stat (incluyendo paréntesis anidados en comm), cálculo de CPU%, construcción de árbol de procesos, reubicación de huérfanos, detección de reuso de PID, transiciones de ciclo de vida.

Limitaciones: Sin acceso real a /proc; solo instantáneas sintéticas. Sin terminal curses. CPU% es una estimación del modelo a partir de deltas de jiffies, no la CPU real del sistema anfitrión.

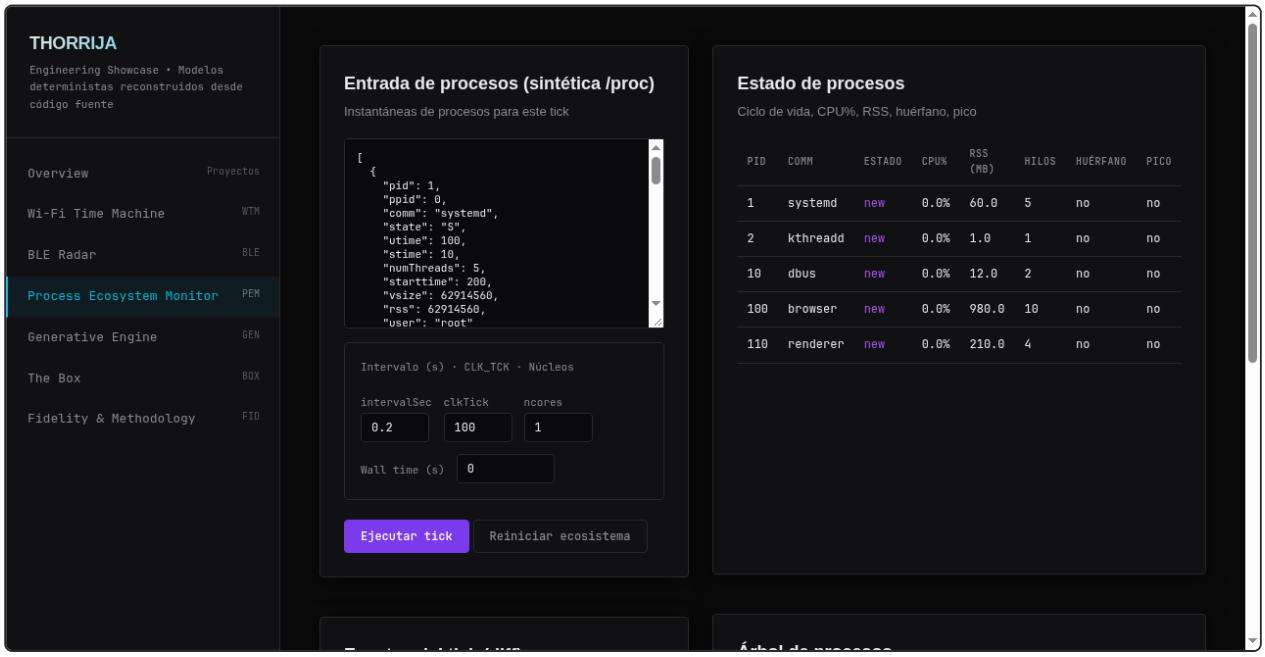


Figura — Laboratorio Process Ecosystem Monitor: entrada sintética de `/proc` y tabla de estado de procesos derivada.

3.4 Terminal Generative Engine

Problema: El arte generativo suele depender de estado aleatorio oculto, lo que hace imposible reproducir un frame específico o compartir un resultado exacto.

Arquitectura: RNG splitmix64-style sin estado → parámetros derivados de semilla → modos de renderizado (flowfield/plasma/attractor) → framebuffer → codificación de medio bloque terminal → salida.

Estructuras de datos: `GenModelInput` (seed, mode, width, height, frame, palette), `GenModelOutput` (stages, final.pixels, final.width, final.height, final.frame, final.seed, final.mode, final.palette, final.cellCount, final.fingerprint).

Algoritmos: RNG splitmix64-style sin estado: cada píxel es una función pura de (semilla, modo, frame, salt, índice). Flowfield: campo de ángulo analítico con partículas trazando líneas de corriente. Plasma: fórmulas de color cerradas por píxel. Atractor: mapa extraño de Clifford/Dejong con sondeo de coeficientes. Huella dactilar del mundo: SHA-256 sobre modo + frame + parámetros + píxeles.

Pruebas: 24 pruebas que cubren RNG sin estado, renderizado de modo, búsqueda de frame O(1), huella dactilar, interpolación de paleta.

Limitaciones: Sin salida terminal/ANSI; no hay exportación de archivo (arrays de píxeles exportados como datos); el motor original tiene 5 pruebas fallantes no reproducidas aquí.

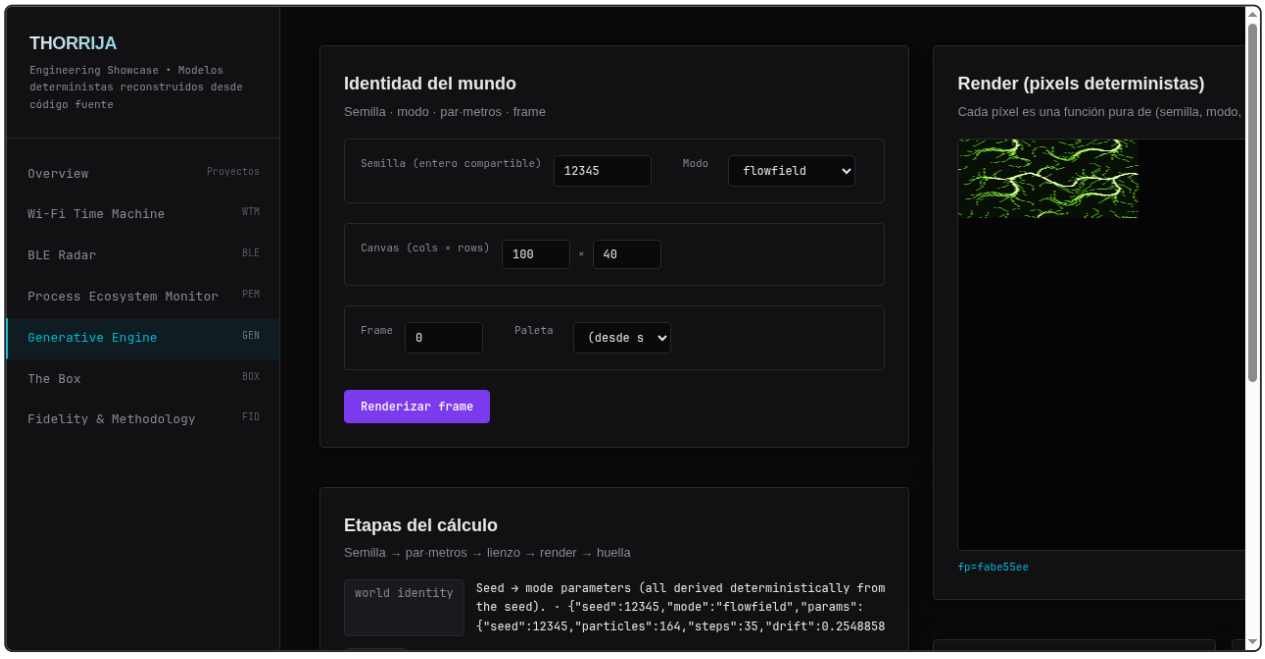


Figura — Laboratorio Terminal Generative Engine: controles de semilla, modo, lienzo y frame, con el render determinista.

3.5 The Box

Problema: La mayoría de los grafos almacenan identidad y linaje como campos explícitos. The Box pregunta: ¿qué pasaría si la identidad y el linaje no se almacenan en absoluto, sino que emergen puramente de reglas de reescritura?

Arquitectura: Sustrato arc-incidencia multi-digraph donde nodos y arcos comparten un mismo espacio de ids, los arcos pueden apuntar a otros arcos (segundo orden). Seis reglas de reescritura (GATHER, BOND, TAG_PROP, QUIET, DESCEND, FUSE) modifican el sustrato. Cuatro políticas del motor (canonical, uniform, weighted, roundrobin). Espacio de

posibilidades: finito, enumerable, sin semilla. Identidad y linaje derivados de etiquetas de arcos, no campos almacenados.

Estructuras de datos: `Nodo` {id, tag, payload (vector de números), born}, `Arco` {id, src, dst, tag, peso, clock}. Espacio de ids compartido entre nodos y arcos. Arcos de segundo orden: un arco puede tener otro arco como extremo.

Algoritmos: Seis reglas de reescritura:

- **GATHER:** Dos partidarios del mismo hub se conectan directamente.
- **BOND:** El ancla y un vecino de salida que comparten dos vecinos de entrada se enlazan.
- **TAG_PROP:** El ancla copia la etiqueta de un vecino de salida.
- **QUIET:** Un nodo que no ha recibido nada en 30 pasos se marca como QUIET (tag 5).
- **DESCEND:** Un nodo QUIET con espacio genera un hijo con su etiqueta y payload con pequeño jitter.
- **FUSE:** Dos hermanos (mismo padre vía DESCEND) con misma etiqueta y payload cercano se identifican; el gemelo se retira; un arco EQUIV queda como testigo de la fusión.

Métricas: degree_tables (grados de entrada/salida, coeficiente de Gini), components (union-find), beta0_beta1 (Euler-exacto sobre grafo de incidencia: beta1 = ‖arcs‖ − ‖objects‖ + beta0), component_size_profile, modularidad (Louvain determinista, un solo nivel), 14 observables nombrados.

Pruebas: 102 pruebas (42 kernel + 25 experimental + 12 validación + 23 validador YAML) que cubren kernel/invariantes, reglas de reescritura, políticas del motor, espacio de posibilidades, identidad/linaje derivado, métricas estructurales, observables nombrados, modelos nulos, p-valores, proveniencia, experimento.

Limitaciones: Fidelidad Nivel 3. BLAKE2b puro TS (no byte-compatible con Python hashlib.blake2b); Float64Array reemplaza NumPy arrays; reglas de foundry hard-coded (sin cargador YAML en el modelo — disponible en the-box-yaml.ts); sin análisis de Koopman, sin puertas clasificadoras (G1–G4), sin batería de modelos nulos en el modelo (la capa experimental la tiene); sin SQLite; sin CLI ni API de store.

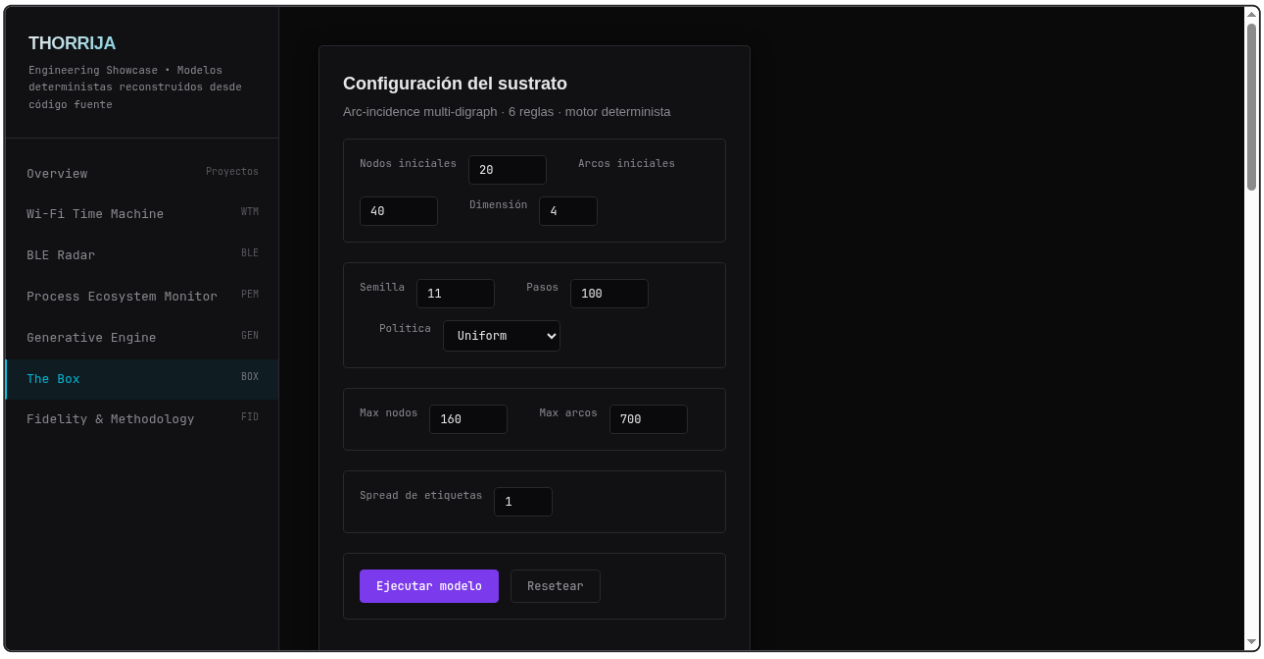


Figura — Laboratorio The Box: configuración del sustrato arc-incidencia — nodos y arcos, dimensión, semilla, pasos, política del motor y spread de etiquetas.

4. Pruebas y Validación

Todos los modelos están verificados por pruebas automatizadas. Total: **182 pruebas, todas aprobadas**.

SUITE	PRUEBAS	UBICACIÓN
Wi-Fi Time Machine	18	tests/wifi-time-machine.test.ts
BLE Radar	16	tests/ble-radar.test.ts
Process Ecosystem Monitor	22	tests/process-ecosystem.test.ts
Terminal Generative Engine	24	tests/generative-engine.test.ts
The Box — kernel/motor/reglas	42	models/the-box.test.ts
The Box — capa experimental	25	models/the-box-experiment.test.ts
The Box — fixtures de validación	12	models/the-box-validation.test.ts
The Box — cargador YAML	23	models/the-box-yaml.test.ts

Objetivo de cobertura: 80%+ en líneas y funciones de cada modelo.

Validación cross-lenguaje: 12 pruebas comparan propiedades estructurales de TypeScript contra valores de referencia de Python construidos a partir de fixtures hechos a mano (no State.random()). Valores equivalentes verificados: conteo de nodos/arcos, distribuciones de grado, beta0/beta1, componentes, modularidad, profundidad de linaje, métricas de identidad, 14 observables nombrados, propiedades de modelos nulos, p-valores.

No byte-compatible: Los hashes de grafo, claves canónicas, unit_hash y huellas de experimento difieren entre Python y TypeScript debido a la implementación de BLAKE2b. Misma semilla produce salida idéntica dentro de cada implementación, pero Python y TypeScript producen hashes diferentes para la misma entrada. Las propiedades estructurales son equivalentes; los valores de hash no lo son.

5. Limitaciones

- **The Box: fidelidad Nivel 3:** BLAKE2b puro TS (no byte-compatible con Python hashlib.blake2b); Float64Array reemplaza NumPy arrays; reglas de foundry hard-coded (sin cargador YAML en el modelo — disponible en the-box-yaml.ts); sin análisis de Koopman; sin puertas clasificadoras (G1–G4); sin batería de modelos nulos en el modelo (capa experimental la tiene); sin SQLite; sin CLI ni API de store.
- **WTM y BLE Radar:** Sin acceso a hardware real; datos sintéticos solo.
- **Terminal Generative Engine:** Sin salida terminal/ANSI; sin exportación de archivo (arrays de píxeles exportados como datos).
- **Process Ecosystem Monitor:** Sin acceso real a /proc; solo instantáneas sintéticas.
- **Todos los modelos:** Sin persistencia SQLite (solo en memoria); sin claves de API; sin servicios externos; todos los datos son sintéticos o calculados desde la entrada del usuario.

6. Líneas de Ingeniería Futura

- Modelos nulos, p-valores y puertas clasificadoras (G1–G4) para la capa experimental de The Box.
- Análisis de Koopman para The Box.
- Capa de persistencia SQLite para The Box.
- CLI y API de store para The Box.
- Exportación de archivo (GIF/PNG/ANSI) para Terminal Generative Engine.
- Acceso a hardware Wi-Fi/BLE en vivo.
- Pruebas E2E de navegador.
- Auditoría de accesibilidad (WCAG 2.2 AA).

7. Contacto y Repositorios

- **Autor:** Thorrija (Rubén N.)
- **Ubicación:** España
- **GitHub:** github.com/ThorrijaDEV
- **Showcase:** github.com/ThorrijaDEV/showcase
- **Modelos:** github.com/ThorrijaDEV/models
- **Portafolio:** github.com/ThorrijaDEV/Portafolio

Todos los modelos son deterministas: misma entrada → misma salida. No se requiere hardware, ni claves de API, ni servicios externos para las demostraciones. Todos los datos mostrados son sintéticos o calculados desde la entrada del usuario — nada está fabricado.

PYEOF